Lecture 36 — Final Review, Year Retrospective & Mock Contest

> Year Curriculum · Prerequisites: Weeks 1–35 · Estimated time: 1 hour review + 5-hour mock contest + 1-hour debrief

1. Overview

This final session has three parts:

1. **Retrospective**: a compact map of everything covered this year, organized by what kind of problem each tool solves, and a guide for recognizing techniques from constraints and keywords.
2. **Mock contest**: a 5-hour, 12-problem ICPC-style contest built from CSES problems spanning the whole year, so it can be graded automatically.
3. **Debrief and next steps**: solution sketches with complexities, the mistakes we expect to see, reference code for the hardest problems, a self-assessment, and a plan for the summer and the Regional track.

2. The year in one map

2.1 Foundations (Weeks 1–9)

- **Judges and I/O** (Week 1), **C++ and STL** (Week 2): fast I/O, `vector`, `sort`, `map`, `set`, `lower_bound`.
- **Complexity and complete search** (Week 3): estimate operations before coding; brute force is both a solution for small inputs and a reference for stress testing.
- **Sorting and greedy** (Week 4): sort by the right key, prove choices with exchange arguments.
- **Binary search** (Week 5), especially **on the answer**: if "can we achieve x?" is monotone in x, search for the boundary.
- **Two pointers** (Week 6): windows whose endpoints only move forward.
- **Prefix sums, 2D prefix sums, difference arrays** (Week 7).
- **Recursion and backtracking** (Week 8): subsets, permutations, pruning.
- **Bit manipulation** (Week 9): masks as sets, enumerating subsets and submasks.

2.2 Math, linear structures and DP basics (Weeks 10–15)

- **Number theory** (Weeks 10–11): gcd, sieve, factorization, modular arithmetic, fast exponentiation, modular inverses, nCr mod p.
- **Stacks, queues, deques** (Week 12): monotonic stack for nearest smaller/greater, monotonic deque for sliding-window extremes.
- **Heaps, ordered sets and maps** (Week 13): `priority_queue`, `multiset`, policy-based order statistics.
- **Dynamic programming** (Weeks 14–15): states, transitions, memoization vs tabulation; knapsack, LIS, LCS, edit distance, grid DP.

2.3 Graphs (Weeks 16–22)

- BFS/DFS, components, bipartiteness, grids (Week 16).
- Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS (Week 17).
- DSU and minimum spanning trees (Week 19).
- Trees: diameter, subtree sizes, tree DP, rerooting (Week 20).
- DAGs, topological sort, DP on DAGs, strongly connected components (Week 21).
- LCA by binary lifting, Euler tour, tree flattening (Week 22).

2.4 Range queries and advanced DP (Weeks 23–27)

- Segment trees (Week 23), Fenwick trees and sparse tables (Week 24), lazy propagation (Week 25).
- Bitmask DP (Week 26); interval and digit DP (Week 27).

2.5 Advanced topics (Weeks 28–35)

- Strings: hashing, prefix function, Z-function (Week 28); tries, Aho-Corasick, suffix arrays (Week 29).
- Combinatorics, inclusion-exclusion, probability, expected value (Week 30).
- Computational geometry: cross products, segments, polygons, convex hull (Week 31).
- Network flow: Edmonds-Karp, Dinic, min cut, bipartite matching (Week 32).
- Game theory: Nim, Sprague-Grundy (Week 33).
- Square-root decomposition and Mo's algorithm (Week 34).
- Contest strategy, debugging, stress testing (Week 35).

3. Recognizing the technique

3.1 Constraints

The input size is the strongest hint (see Week 35 for the full table): $n \le 20$ suggests exponential subset algorithms; $n \le 500$, cubic; $n \le 5000$, quadratic; $n \le 2 \cdot 10^5$, $O(n \log n)$ or $O(n\sqrt{n})$; values up to 10^{18}, logarithmic or square-root math.

3.2 Keywords

Statement says	Usually means
"minimize the maximum" / "maximize the minimum"	binary search on the answer + greedy check
"longest/shortest subarray such that"	two pointers, or prefix sums + map
"count ways modulo 10^9+7"	DP or combinatorics (factorials, inclusion-exclusion)
"$n \le 20$", "visit every city"	bitmask DP
"range queries with updates"	Fenwick / segment tree; lazy for range updates
"offline queries about distinct values"	Mo's algorithm or sort-by-right-end + BIT
"next greater", "largest rectangle"	monotonic stack
"minimum moves on a grid"	BFS (multi-source, 0-1 BFS, state = position + extra info)
"roads with lengths, shortest"	Dijkstra (Bellman-Ford if negative)
"connect everything cheaply"	MST
"prerequisites", "order of tasks"	topological sort; SCC if cycles
"path between two nodes of a tree"	LCA, root-path prefix values
"each person gets at most one item"	bipartite matching / flow
"minimum number of roads to block"	min cut
"players alternate", "optimal play"	game DP, Nim, Grundy numbers
"expected number of"	linearity of expectation, expected-value DP
"occurrences of many words"	Aho-Corasick (or hashing per length)
"number of distinct substrings"	suffix array + LCP
"points, polygon, convex"	cross products, shoelace, convex hull

Keywords are hints, not proofs: always confirm that the structure the technique needs (monotonicity, optimal substructure, independence) is actually present.

3.3 The method

From Week 18, the loop we have practiced all year:

1. **Understand**: restate the task in one sentence; identify input, output, and constraints.
2. **Examples**: work through the samples and a few hand-made tiny cases.
3. **Brute force**: write down the obvious slow solution. Keep it: it becomes your stress tester.
4. **Observe**: look for monotonicity, independence, small parameters, invariants, symmetry.
5. **Optimize**: choose the data structure or technique that removes the bottleneck of the brute force.
6. **Verify**: complexity, edge cases, and a stress test before (or right after) submitting.

4. The mock contest

4.1 Rules

- 5 hours, ICPC scoring: number of problems solved, then penalty (minutes to acceptance plus 20 per rejected attempt, solved problems only).
- Teams of three with one computer, or individually as extra practice.
- Printed notebook and C++ reference allowed; no internet search beyond the judge.
- Problems are **not** sorted by difficulty. Triage first (Week 35).

4.2 Problem set

All problems are from the CSES Problem Set, so you can submit them for automatic judging.

	Problem	Link
A	Concert Tickets	https://cses.fi/problemset/task/1091
B	Hamiltonian Flights	https://cses.fi/problemset/task/1690
C	Sum of Two Values	https://cses.fi/problemset/task/1640
D	Company Queries II	https://cses.fi/problemset/task/1688
E	Prime Multiples	https://cses.fi/problemset/task/2185
F	Dice Combinations	https://cses.fi/problemset/task/1633
G	Download Speed	https://cses.fi/problemset/task/1694
H	String Matching	https://cses.fi/problemset/task/1753
I	Road Reparation	https://cses.fi/problemset/task/1675
J	Nim Game I	https://cses.fi/problemset/task/1730
K	Dynamic Range Sum Queries	https://cses.fi/problemset/task/1648
L	Shortest Routes I	https://cses.fi/problemset/task/1671

Stop reading here until the contest is over.

5. Debrief: solution sketches

Easy tier

C — Sum of Two Values (Weeks 4, 6). Find two positions with a[i] + a[j] = x. Sort (value, original index) pairs and run two pointers from both ends, or scan with a hash map from value to index, checking x − a[i] before inserting a[i]. O(n log n). Pitfall: using the same element twice; `unordered_map` can be attacked on some judges (use `map` or a custom hash).

F — Dice Combinations (Week 14). dp[0] = 1, dp[s] = Σ_{d=1..6} dp[s − d] mod 10^9+7. O(6n).

J — Nim Game I (Week 33). First player wins iff the XOR of the heaps is non-zero. O(n) per test.

H — String Matching (Week 28). Count occurrences of p in t: prefix function of p + '#' + t and count positions where π equals |p|, or the Z-function. O(|t| + |p|). Hashing works too, but beware of anti-hash tests with a fixed base.

Medium tier

A — Concert Tickets (Week 13). For each customer with maximum price x, sell the most expensive ticket ≤ x. Keep the prices in a `multiset`; `it = upper_bound(x)`; if `it == begin()` print −1, else step back, print and `erase(it)` (the iterator, not the value, or all copies vanish). O((n + m) log n).

L — Shortest Routes I (Week 17). Dijkstra from city 1 with `long long` distances (up to $10^5 \cdot 10^9$) and the stale-entry check `if (d > dist[v]) continue;`. O(m log n).

I — Road Reparation (Week 19). Kruskal: sort edges by cost and union with DSU; the answer is the total cost, or IMPOSSIBLE if fewer than n − 1 edges were added. Cost sum needs `long long`. O(m log m).

K — Dynamic Range Sum Queries (Week 24). Fenwick tree; for "set a[k] = u" add u − a[k] and update the stored a[k]. O((n + q) log n).

E — Prime Multiples (Week 30). Inclusion-exclusion over the 2^k subsets of the k ≤ 20 primes; a subset contributes ±⌊n / product⌋, and the product must be computed without overflow (stop when it exceeds n). O($2^k \cdot k$).

Hard tier

D — Company Queries II (Week 22). LCA in a rooted tree with n, q ≤ $2 \cdot 10^5$, where the boss of employee i is a number smaller than i. Build `up[j][v]` = 2^j-th ancestor. Because parents have smaller labels, iterating v in increasing order computes depths without recursion. Query: lift the deeper node to equal depth, then for j from high to low lift both while `up[j][a] != up[j][b]`. O((n + q) log n).

B — Hamiltonian Flights (Week 26). Count routes from city 1 to city n that visit every city exactly once (n ≤ 20). dp[mask][v] = number of routes starting at 1, visiting exactly the cities in `mask`, ending at v. Only masks containing city 1 matter, and city n may only be entered as the last city (when the mask becomes full), which prunes many states. Transition along each outgoing flight of v. O($2^n \cdot m$) with m ≤ n^2; use `int` storage (taking mod after each addition) to keep the table at $2^{20} \cdot 20 \cdot 4$ bytes ≈ 84 MB.

G — Download Speed (Week 32). Maximum flow from 1 to n; Dinic with `long long` capacities. n ≤ 500, m ≤ 1000 makes even Edmonds-Karp fast enough.

What usually goes wrong

- E: forgetting the overflow check, or the wrong sign for even-size subsets.
- B: iterating all (v, u) pairs instead of the adjacency list, a `long long` DP table exceeding memory, or allowing city n in the middle of a route.
- L, I, G: `int` where sums need 64 bits.
- A: `multiset::erase(value)` removing all equal tickets.
- D: building ancestor tables in an order where `up[j-1][v]` of the needed node is not yet known.
- Every problem: not resetting state when submitting multiple test-case problems (J).

6. Reference implementations

6.1 A — Concert Tickets

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    multiset<int> tickets;
    for (int i = 0; i < n; i++) {
        int h;
```

```
        cin >> h;
        tickets.insert(h);
    }
    for (int i = 0; i < m; i++) {
        int t;
        cin >> t;
        auto it = tickets.upper_bound(t);
        if (it == tickets.begin()) {
            cout << -1 << "\n";
        } else {
            --it;
            cout << *it << "\n";
            tickets.erase(it);  // erase one copy only
        }
    }
}
```

Sample: `5 3 / 5 3 7 8 5 / 4 8 3` → 3, 8, −1.

6.2 D — Company Queries II (binary lifting LCA)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    const int LOG = 18;  // 2^18 > 2 * 10^5
    vector<array<int, LOG>> up(n + 1);
    vector<int> depth(n + 1, 0);
    up[1].fill(1);  // the root is its own ancestor
    for (int v = 2; v <= n; v++) {
        int boss;
        cin >> boss;
        up[v][0] = boss;
        depth[v] = depth[boss] + 1;  // boss < v, so depth[boss] is known
    }
    for (int j = 1; j < LOG; j++)
        for (int v = 1; v <= n; v++) up[v][j] = up[up[v][j - 1]][j - 1];
    auto lca = [&](int a, int b) {
        if (depth[a] < depth[b]) swap(a, b);
        int diff = depth[a] - depth[b];
        for (int j = 0; j < LOG; j++)
            if (diff >> j & 1) a = up[a][j];
        if (a == b) return a;
        for (int j = LOG - 1; j >= 0; j--)
            if (up[a][j] != up[b][j]) { a = up[a][j]; b = up[b][j]; }
        return up[a][0];
    };
    while (q--) {
        int a, b;
```

```
        cin >> a >> b;
        cout << lca(a, b) << "\n";
    }
}
```

Note the loop order: level j is built for all vertices after level j − 1 is complete, so the order of vertices does not matter there. Sample: `5 3 / 1 1 3 3 / 4 5 / 2 5 / 1 4` → 3, 1, 1.

6.3 B — Hamiltonian Flights (bitmask DP)

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1'000'000'007;
static int dp[1 << 20][20];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a - 1].push_back(b - 1);
    }
    int full = (1 << n) - 1;
    dp[1][0] = 1;  // at city 0 having visited {0}
    for (int mask = 1; mask <= full; mask += 2) {      // masks containing city 0
        if ((mask >> (n - 1) & 1) && mask != full) continue;  // n-1 only at the end
        for (int v = 0; v < n; v++) {
            int cur = dp[mask][v];
            if (!cur) continue;
            for (int u : adj[v]) {
                if (mask >> u & 1) continue;
                int nm = mask | (1 << u);
                dp[nm][u] += cur;
                if (dp[nm][u] >= MOD) dp[nm][u] -= MOD;
            }
        }
    }
    cout << dp[full][n - 1] << "\n";
}
```

Sample: `4 6 / 1 2 / 1 3 / 2 3 / 3 2 / 2 4 / 3 4` → 2 (routes 1→2→3→4 and 1→3→2→4). For n = 1 the answer is 1 (the empty route), which the code also produces since `full = 1` and `dp[1][0] = 1`.

6.4 I — Road Reparation (Kruskal + DSU)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<array<long long, 3>> edges(m);  // {cost, a, b}
    for (auto& e : edges) cin >> e[1] >> e[2] >> e[0];
    sort(edges.begin(), edges.end());
    DSU dsu(n + 1);
    long long total = 0;
    int used = 0;
    for (auto& e : edges)
        if (dsu.unite((int)e[1], (int)e[2])) { total += e[0]; used++; }
    if (used == n - 1) cout << total << "\n";
    else cout << "IMPOSSIBLE\n";
}
```

Sample: `5 6 / 1 2 3 / 2 3 5 / 2 4 2 / 3 4 8 / 5 1 7 / 5 4 4` → 14.

The remaining problems are covered by reference code in their own lectures: Dice Combinations (Week 14), Shortest Routes I (Week 17), Dynamic Range Sum Queries (Week 24), String Matching (Week 28), Prime Multiples (Week 30), Download Speed (Week 32), Nim Game I (Week 33).

7. The year's most common mistakes

1. **Integer overflow**: sums of 10^5 values of 10^9, products of two 10^9 numbers, `1 << 40`.
2. **Multi-test state** not reset; reading interrupted by an early `break`.
3. **Off-by-one** in ranges, loop bounds, and binary search termination.

4. **Deep recursion** on path-shaped trees and graphs (convert to iterative or use an explicit stack).
5. **INF too small** for 64-bit distances; adding to INF and overflowing.
6. **Dijkstra without stale-entry skipping**, or used with negative edges.
7. **Graph construction**: forgetting reverse edges (undirected graphs, residual graphs).
8. **Negative modulo** results in C++.
9. **Unproven greedy** choices; always test against a brute force.
10. **Output format**: capitalization, separators, floating-point precision.

8. Self-assessment and next steps

After the contest and the upsolving, rate yourself from 1 to 5 on whether you can write each of the following from memory, correctly, in about 15 minutes: BFS and Dijkstra; DSU and Kruskal; segment tree with lazy propagation; Fenwick tree; LIS in O(n log n); bitmask DP; LCA with binary lifting; KMP or Z-function; Aho-Corasick; Dinic; convex hull; Nim/Grundy computation; Mo's algorithm. Anything rated below 4: re-solve that week's practice list and add the code to your notebook.

Next stage. The Regional track (02-Regionals) focuses on ICPC regional difficulty: implementation-heavy problems, advanced greedy (exchange arguments, regret heaps), graph modeling with state spaces, DP optimizations, sweep lines and coordinate compression, number theory II (CRT, Euler's phi, Möbius), geometry and strings at regional level, and a full mock regional. Later tracks (Nationals, World Finals) go to persistent structures, heavy-light and centroid decomposition, min-cost flow, suffix automata, FFT, and beyond.

Summer plan. Each week: one or two virtual contests (Codeforces Div. 2, AtCoder ABC/ARC, or an ICPC gym set as a team), upsolve every problem you were close to, 5–10 topic problems from this year's lists, and one new tested template in the notebook. Keep a log of problems and mistakes and skim it before each contest. The largest gains come from volume and honest upsolving, not from learning yet another algorithm.

9. Practice problems

The 12 mock-contest problems above, plus these mixed review sets:

- CSES Problem Set (https://cses.fi/problemset) — finish the sections Sorting and Searching, Dynamic Programming, Graph Algorithms, Range Queries, Tree Algorithms
- AtCoder Educational DP Contest (https://atcoder.jp/contests/dp) — 26 DP problems, A–Z
- Codeforces 1520 (Div. 3) (https://codeforces.com/contest/1520) — a full contest for timing practice
- Codeforces 1324 (Div. 3) (https://codeforces.com/contest/1324) — a second timing contest
- Kattis — any recent NWERC or NAC problem set, as a team, under contest conditions

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook* (free PDF) — the whole book is a good review of this year
- Steven Halim, Felix Halim, Suhendry Effendy, *Competitive Programming 4*, Books 1 and 2
- cp-algorithms (https://cp-algorithms.com) — reference for every algorithm in this curriculum
- USACO Guide (https://usaco.guide) — topic-organized practice with problem lists by difficulty
- KACTL (https://github.com/kth-competitive-programming/kactl) — a model team notebook to compare yours against